

# Cache Trace 评估报告

样本: /Users/zhuhanyuan/Documents/XiaoBa-CLI/logs/cache-trace/2026-07-31 · 生成时间: 2026-07-31

**结论:** 不能只凭“加载前/连续调用”判断。应以实际响应的 `cache_read_tokens` 与 `cache_hit_ratio` 判定结果，以每次实际序列化并发出的 `context` 拼装判定原因。此样本总体有效但不稳定：加权缓存命中 69.5%，存在明显的历史消息改写或前缀断裂问题。

## 1. 评分与摘要

|                                              |                                                |                                                         |
|----------------------------------------------|------------------------------------------------|---------------------------------------------------------|
| <div>68/100</div> <div>总体评价：可用，但未达稳定高效</div> | <div>69.5%</div> <div>按输入 token 加权的缓存命中率</div> | <div>36.4M</div> <div>缓存读取 token，占 52.4M 输入 token</div> |
|----------------------------------------------|------------------------------------------------|---------------------------------------------------------|

| 维度       | 评分 | 判断                                                                                                                  |
|----------|----|---------------------------------------------------------------------------------------------------------------------|
| 实际缓存效果   | 70 | 整体加权命中 69.5%，中位单次命中 76.2%，说明稳定前缀通常能复用。                                                                              |
| 上下文前缀稳定性 | 62 | 系统提示与工具定义在可关联调用中均保持一致，但仍有 210 次命中低于 50%。问题主要落在消息历史序列。                                                               |
| 可观测性     | 80 | 已有 usage、system/tool hash 和 message diff，可做归因；但 65 个连续对没有 previous_run_id，且缺少请求级 cache key、首 token 前缀 hash 和部署路由信息。 |
| 模型与会话隔离  | 58 | 同一会话内有模型段切换。缓存评估必须按 provider、实际 model/deployment、cache key 分段；混合平均会误判冷启动和跨缓存域。                                      |
| 异常恢复能力   | 68 | 多数连续链路表现健康，但有 9 次稳定系统与工具下的零命中，且部分样本已有很长消息前缀仍零命中。                                                                    |

## 2. 这次应如何评估

**先看结果，再看拼装。**实际 `cache_read_tokens` 和 `cache_hit_ratio` 是“是否命中”的唯一结果证据；`context` 拼装、哈希和相邻 diff 是“为什么命中或失效”的解释证据。两者不能互相替代。

- 连续调用不是天然可缓存。只有请求起始 token 序列保持一致，才可能复用已缓存前缀。
- 把静态系统指令、固定工具定义放最前面；把用户当轮输入、动态计划、工具结果和时间戳放后面。
- 模型、实际部署/路由和 cache key 是分段键。切换其中任一项，应作为预期冷启动而非拼装失败。
- 系统和 tools hash 相同不足以证明可命中；历史消息若被截断、重排、压缩或改写，首个差异位置之前以后的缓存会失去价值。

**官方规则的应用边界:** 本报告以 OpenAI Prompt Caching 指南所述的自动前缀缓存、稳定前缀和 cache key 分路由原则作评价框架。由于本次官方文档检索网关不可用，未将网页抓取结果作为额外证据；数值均直接来自本地 trace 的 `response_usage`。

### 3. Trace 数据结果

| 范围                   | 调用数   | 加权命中  | 中位命中  | 说明                                       |
|----------------------|-------|-------|-------|------------------------------------------|
| 全量                   | 1,082 | 69.5% | 76.2% | 52.37M 输入 token，36.42M cache read token。 |
| 可关联且 system/tools 稳定 | 936   | 69.6% | —     | 其中 210 次低于 50%，9 次零命中；这部分才是重点问题池。        |
| DeepSeek-V4-Pro 段    | 758   | 72.0% | —     | 可关联且 system/tools 稳定的调用。                 |
| GPT-5.6-Terra 段      | 178   | 57.5% | —     | 同一口径下显著偏低，并包含全部 9 次零命中。                  |

### 4. 连续调用的定位证据

| 消息前缀情况           | 样本  | 加权命中  | 低于 50% | 解读                                   |
|------------------|-----|-------|--------|--------------------------------------|
| 首个差异在前 5 条消息内    | 237 | 68.6% | 83     | 前缀早断最常见，通常对应 branch、摘要替换、重放历史或上下文重构。 |
| 首个差异在第 5 至 19 条  | 230 | 58.9% | 54     | 已影响可复用主体，优先查旧 tool result 的规范化和截断。   |
| 首个差异在第 20 至 99 条 | 128 | 82.0% | 10     | 表现最好，符合“静态前缀在前、追加动态尾部”的目标形态。         |
| 改写 6 条或以上历史消息    | 26  | 25.2% | 22     | 强烈负相关，是最明确的可修复失效模式。                  |

关键反例：cc\_group:grp\_1176 的 GPT-5.6-Terra 段曾出现已有 142 和 156 条相同前缀却 0% 命中。这不是简单的“加载前冷启动”；应追查 cache key、真实部署路由、缓存 TTL/隔离域，以及序列化后 token 前缀是否确实完全相同。

## 5. 根因优先级与改造建议

### P0：把“实际出站请求”作为唯一拼装真相

- 在发送 API 前生成 canonical request fingerprint：model/deployment、prompt\_cache\_key、system、tools、messages 的确定性序列化哈希。
- 记录 token 级或分块级前缀 hash，并标记第一个 token/消息差异，而不仅是总 message SHA。
- 动态字段不得混入稳定 system 区：时间、plan、subagent 状态、runner hint、请求 ID 均应放到动态尾部或从 prompt 移除。

### P1：禁止历史消息的反复重写

- 工具结果截断/压缩使用一次性、确定性、幂等的规范化；相同输入必须产生字节级相同输出。
- 不要为新一轮请求重排、重包装或重新格式化已存在的 history。新增内容只追加到尾部。
- 把分支、摘要切换、模型切换显式标成新 episode；不要把它们当作同一条连续缓存链路。

### P2：补足观测字段，区分“拼装问题”和“提供方未复用”

- 每条 trace 增加：prompt\_cache\_key、实际 endpoint、模型 deployment/路由、请求时间、缓存阈值是否满足、response id 和原始 usage。
- 低命中自动分类：预期冷启动、system/tool 变化、首消息前缀断裂、历史改写、长前缀仍未命中、不可判定。
- 65 条缺 previous\_run\_id 的连续对必须标为不可判定，不得计入拼装失败率。

## 6. 验收目标

| 指标                    | 当前        | 建议门槛        | 验收口径                              |
|-----------------------|-----------|-------------|-----------------------------------|
| 稳定 system/tools 的加权命中 | 69.6%     | $\geq 80\%$ | 按模型、cache key、部署分别计算；剔除首调用。       |
| 稳定链路低命中               | 210 / 936 | $\leq 10\%$ | cache_hit_ratio 小于 50% 的占比。       |
| 历史改写 6 条以上            | 26 次      | 接近 0        | 仅允许显式 compaction episode；其他调用只追加。 |
| 连续性不可判定               | 65 对      | 0           | 每一条非首调用都有正确 previous_run_id。      |

## 7. 最终判断

应当“两层评估”：第一层用 provider 返回的 cache\_read\_tokens/cache\_hit\_ratio 判定好坏；第二层针对低命中，分析实际 context 拼装后的稳定前缀、工具定义、历史消息与 cache key。就这份 trace 而言，系统和工具稳定不是主要矛盾；历史消息前缀断裂、跨模型/路由混合以及观测缺口才是主因。

数据来源：本地 trace 1,082 个 JSON 文件；统计基于 response\_usage.input\_tokens、cache\_read\_tokens、cache\_hit\_ratio 和 diff 字段。模型名称按 trace 原样展示；本报告不将代理路由名称等同于 OpenAI 官方模型部署。